



Data Structure

(Course Code: 25B11CI318)

Multi-List and Sparse Matrix

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Multilist?

Definition

A multilist is a linked structure in which the same record can participate in more than one linked list, using separate link fields for different relationships.

- Store the record once.
- Provide multiple ways to reach or traverse it.
- Each link field belongs to a particular list.

Example

A registration record belongs to both a student's course list and a course's student list.

Why Use a Multilist?

Suppose a system must answer:

- ① Which courses does a student take?
- ② Which students take a particular course?

A multilist maintains both relationships.

Shared Record

The same enrollment node stores the student-course pair and links into both relevant lists.

Updating shared data changes one record rather than several duplicated copies.

Singly, Doubly and Multiply Linked Lists

Structure	Link Meaning	Purpose
Singly linked list	Next item in one sequence	Forward traversal
Doubly linked list	Previous and next in the same sequence	Bidirectional traversal
Multilist	Links for different sequences or relationships	Multiple views of shared records

Terminology

A multilevel list uses child links to represent hierarchy. A multilist here represents shared membership in several lists.

Student-Course Registration Example

Consider these enrollments:

$(S_1, DS), (S_1, DA), (S_2, DS), (S_3, DA).$

Student View

$S_1 : DS, DA$

$S_2 : DS$

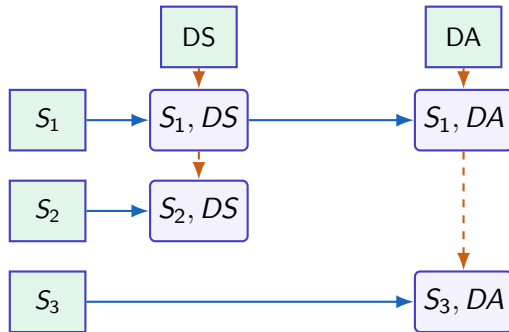
$S_3 : DA$

Course View

$DS : S_1, S_2$

$DA : S_1, S_3$

Multilist Diagram



Solid blue: same student.

Dashed orange: same course.

Multilist Node in C

```
1 typedef struct Enrollment {  
2     int studentId;  
3     int courseId;  
4  
5     struct Enrollment *nextForStudent;  
6     struct Enrollment *nextForCourse;  
7 } Enrollment;
```

- Follow nextForStudent to list a student's courses.
- Follow nextForCourse to list a course's students.

Each enrollment node appears in both traversals.

Multilist Insertion and Deletion

Insertion

Create one record, then link it into every relevant list. Apply duplicate and ordering rules consistently.

Deletion

Locate the record, unlink it from every list containing it, then release its memory exactly once.

Common Error

Freeing a record while another list still points to it creates a dangling pointer.

Sparse Matrices

Storing Nonzero Entries Efficiently

What is a Sparse Matrix?

Definition

A sparse matrix is a matrix in which relatively few entries are nonzero.

For an $m \times n$ matrix, let:

t = number of nonzero entries.

Its density is:

$$\text{Density} = \frac{t}{mn}.$$

Its zero fraction is:

$$\text{Zero fraction} = 1 - \frac{t}{mn}.$$

There is no universal density threshold for choosing sparse storage.

Running Example

$$A = \begin{bmatrix} 0 & 0 & 7 & 0 & 0 \\ 5 & 0 & 0 & 0 & 9 \\ 0 & 0 & 0 & -2 & 0 \\ 0 & 6 & 0 & 0 & 3 \end{bmatrix}$$

- Rows: $m = 4$.
- Columns: $n = 5$.
- Total entries: $mn = 20$.
- Nonzero entries: $t = 6$.

$$\text{Density} = 6/20 = 30\%.$$

Dense Representation

A conventional two-dimensional array stores every entry:

$$A[0 \dots m - 1][0 \dots n - 1].$$

- Space: $\Theta(mn)$ values.
- Access to a known position: $O(1)$.
- Zeros occupy storage like nonzero entries.

Example

The running matrix stores 20 values in dense form, although only 6 are nonzero.

Sparse Representation Choices

- ① Triplet or coordinate representation.
- ② A linked list of nonzero entries.
- ③ Separate row lists.
- ④ Orthogonal row and column lists.
- ⑤ Compressed Sparse Row: CSR.
- ⑥ Compressed Sparse Column: CSC.

Choice Depends on Operations

Construction, updates, row traversal and column traversal have different requirements.

Triplet Representation

Store each nonzero entry as:

$(row, column, value)$.

For the running example:

$(0, 2, 7), (1, 0, 5), (1, 4, 9),$
 $(2, 3, -2), (3, 1, 6), (3, 4, 3).$

Convention

Use zero-based indices and sort entries first by row, then by column.

Matrix dimensions and the number of entries are metadata.

Triplet Table with Metadata

Record	Row	Column	Value
Header	4	5	6
1	0	2	7
2	1	0	5
3	1	4	9
4	2	3	-2
5	3	1	6
6	3	4	3

Header Meaning

The first record represents (*rows*, *columns*, *nonzeros*). It is not a matrix entry.

Triplet Representation in C

```
1 typedef struct {  
2     int row;  
3     int col;  
4     int value;  
5 } Term;  
6  
7 Term entries[] = {  
8     {0, 2, 7},  
9     {1, 0, 5},  
10    {1, 4, 9},  
11    {2, 3, -2},  
12    {3, 1, 6},  
13    {3, 4, 3}  
14 };  
15  
16 int rows = 4, cols = 5, nonzeros = 6;
```

Canonical Sparse Representation

For the algorithms in these slides:

- Store only nonzero values.
- Store at most one entry for each coordinate.
- Keep indices within matrix dimensions.
- Keep triplets in row-major coordinate order.

When an Operation Produces Zero

Remove or omit that entry. For example, $5 + (-5) = 0$ requires no stored result node.

Input formats may allow duplicates, but they should be combined before using these algorithms.

Triplet Representation: Costs

Operation	Cost
Store all nonzeros	$\Theta(t)$
Scan all nonzeros	$\Theta(t)$
Search an unsorted triplet array	$O(t)$
Binary search a sorted coordinate array	$O(\log(t + 1))$
Insert into a packed sorted array	$O(t)$ worst case
Convert from a dense matrix	$\Theta(mn)$

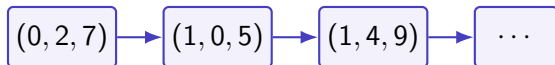
Trade-Off

Triplets are simple and compact, but insertion may require shifting array entries.

A Single Linked List of Nonzeros

Each node stores:

$(row, column, value, next)$.



- Dynamic allocation avoids array shifting.
- Finding a position can still take $O(t)$ time.
- Column traversal may require scanning the entire list.

Orthogonal Linked-List Representation

Definition

An orthogonal representation stores each nonzero entry in both a row list and a column list.

Each entry has two independent links:

- right: next nonzero in the same row.
- down: next nonzero in the same column.

One node participates in two lists.

This is a direct application of a multilist.

Entry and Header Structure

Each nonzero node stores:

row, col, value, right, down.

Maintain:

rowHead[0 . . . *m* - 1],

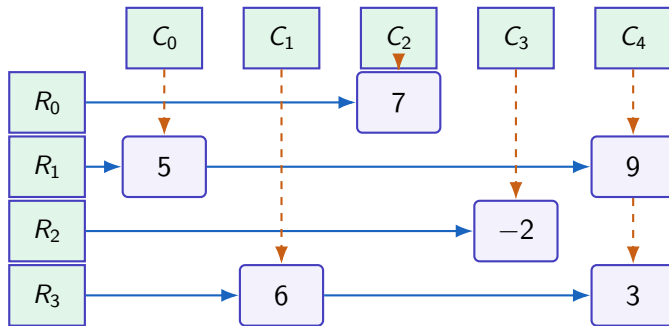
colHead[0 . . . *n* - 1].

- Row lists are sorted by increasing column.
- Column lists are sorted by increasing row.
- Empty lists have a NULL head.

Convention

The implementation here uses NULL-terminated lists. Circular header-node variants are also possible.

Orthogonal Representation of the Example



Solid blue: right links. Dashed orange: down links.

Terminal NULL links are omitted. Crossings are not nodes.

Reading a Row

To list row 1:

$rowHead[1] \longrightarrow (1, 0, 5) \longrightarrow (1, 4, 9) \longrightarrow NULL.$

Thus:

$$A[1, *] = [5, 0, 0, 0, 9].$$

Cost

If row i has r_i nonzero entries, listing only those entries takes:

$$\Theta(r_i + 1).$$

Printing all n positions of the row still requires $\Theta(n)$ output work.

Reading a Column

To list column 4:

$colHead[4] \longrightarrow (1, 4, 9) \longrightarrow (3, 4, 3) \longrightarrow NULL.$

Thus:

$$A[*, 4] = \begin{bmatrix} 0 \\ 9 \\ 0 \\ 3 \end{bmatrix}.$$

If column j has c_j nonzeros:

List its nonzeros in $\Theta(c_j + 1)$.

Searching for an Entry

To find $A[i, j]$ using row i :

- 1 Start at $rowHead[i]$.
- 2 Follow right links while the column is below j .
- 3 If column j is found, return its value.
- 4 Otherwise return zero.

$$\text{Search cost} = O(r_i + 1)$$

Alternative

Searching the column costs $O(c_j + 1)$. If row and column counts are maintained, choose the shorter list.

Inserting a Nonzero Entry

To insert (i, j, v) where $v \neq 0$:

- 1 Validate the coordinates.
- 2 Search row i for the correct column position.
- 3 If the coordinate already exists, update its value.
- 4 Otherwise search column j for the row position.
- 5 Allocate one node.
- 6 Link it into both lists.

$$\text{New-entry insertion} = O(r_i + c_j + 1).$$

Insertion Example: Set $A[1, 2] = 4$

Before:

Row 1: $(1, 0, 5) \longrightarrow (1, 4, 9),$

Column 2: $(0, 2, 7).$

After:

Row 1: $(1, 0, 5) \longrightarrow \boxed{(1, 2, 4)} \longrightarrow (1, 4, 9),$

Column 2: $(0, 2, 7) \longrightarrow \boxed{(1, 2, 4)}.$

Shared Node

Both lists reference the same newly allocated node. The nonzero count increases from 6 to 7.

Deleting an Entry

To remove the entry at (i, j) :

- 1 Locate it in row i .
- 2 Locate its predecessor link in column j .
- 3 Unlink it from the row list.
- 4 Unlink it from the column list.
- 5 Free it once and decrease the nonzero count.

Assigning Zero

Setting an existing entry to zero performs deletion. Setting an absent entry to zero leaves the matrix unchanged.

$$\text{Deletion} = O(r_i + c_j + 1).$$

Deleting an Entry: Example

Remove (1, 4, 9) from the original matrix.

Row 1 changes from:

$$(1, 0, 5) \longrightarrow (1, 4, 9)$$

to:

$$(1, 0, 5).$$

Column 4 changes from:

$$(1, 4, 9) \longrightarrow (3, 4, 3)$$

to:

$$(3, 4, 3).$$

$$\boxed{t : 6 \longrightarrow 5}.$$

Orthogonal Lists: Storage and Costs

Property or Operation	Cost
Nodes and header arrays	$\Theta(t + m + n)$ space
Search through row i	$O(r_i + 1)$
New-entry insertion	$O(r_i + c_j + 1)$
Deletion	$O(r_i + c_j + 1)$
Traverse all row lists	$\Theta(m + t)$
Traverse all column lists	$\Theta(n + t)$

Pointer Updates

Once predecessor positions are known, linking or unlinking requires only $O(1)$ pointer changes.

Efficient Construction from Sorted Triplets

Suppose triplets are sorted by row, then column.

Maintain a tail pointer for every row and column.

For each triplet:

- 1 Allocate one entry node.
- 2 Append it to its row using the row tail.
- 3 Append it to its column using the column tail.

Because input rows increase, each column also receives entries in increasing row order.

$$\text{Construction time} = O(m + n + t).$$

Sparse Storage Has Overhead

Sparse structures store positions and links in addition to values.

- Triplets store two indices per value.
- Orthogonal nodes also store two pointers.
- Header arrays require extra storage.
- Structure padding may increase node size.

Important

A small or moderately dense matrix may use less memory as a dense array than as a linked sparse matrix.

The running example illustrates structure rather than guaranteeing memory savings.

C Implementation: Data Structure

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define ROWS 4
5 #define COLS 5
6
7 typedef struct Entry {
8     int row, col, value;
9     struct Entry *right;
10    struct Entry *down;
11 } Entry;
12
13 typedef struct {
14     Entry *rowHead[ROWS];
15     Entry *colHead[COLS];
16     int nonzeros;
17 } Sparse;
```

This classroom implementation uses fixed matrix dimensions.

C Implementation: Entry Lookup

```
1 int getValue(const Sparse *s, int r, int c)
2 {
3     /* Caller supplies valid coordinates. */
4     const Entry *p = s->rowHead[r];
5
6     while (p != NULL && p->col < c)
7         p = p->right;
8
9     if (p != NULL && p->col == c)
10         return p->value;
11
12     return 0;
13 }
```

An absent node represents a zero entry.

Pointer-to-Pointer Links

The update routines use a pointer to a link:

Entry ****link**.

- Initially, it may point to a row or column head.
- Later, it may point to a node's right or down field.
- Assigning through it updates the actual link.

Benefit

The same code handles insertion or deletion at the head and in the middle of a list.

C Implementation: Remove an Entry

```
1 void eraseEntry(Sparse *s, int r, int c)
2 {
3     /* Caller supplies valid coordinates. */
4     Entry **rp = &s->rowHead[r];
5     while (*rp != NULL && (*rp)->col < c)
6         rp = &(*rp)->right;
7
8     if (*rp == NULL || (*rp)->col != c)
9         return;
10
11     Entry *p = *rp;
12     Entry **cp = &s->colHead[c];
13     while (*cp != p)
14         cp = &(*cp)->down;
15
16     *rp = p->right;
17     *cp = p->down;
18     free(p);
```

C Implementation: Set an Entry, Part 1

```
1 int setValue(Sparse *s, int r, int c, int value)
2 {
3     if (r < 0 || r >= ROWS || c < 0 || c >= COLS)
4         return 0;
5
6     if (value == 0) {
7         eraseEntry(s, r, c);
8         return 1;
9     }
10
11     Entry **rp = &s->rowHead[r];
12     while (*rp != NULL && (*rp)->col < c)
13         rp = &(*rp)->right;
14
15     if (*rp != NULL && (*rp)->col == c) {
16         (*rp)->value = value;
17         return 1;
18     }
```

C Implementation: Set an Entry, Part 2

Continue the function:

```
1  Entry **cp = &s->colHead[c];  
2  while (*cp != NULL && (*cp)->row < r)  
3      cp = &(*cp)->down;  
4  
5  Entry *p = malloc(sizeof *p);  
6  if (p == NULL)  
7      return 0;  
8  
9  p->row = r;  
10 p->col = c;  
11 p->value = value;  
12 p->right = *rp;  
13 p->down = *cp;  
14  
15 *rp = p;  
16 *cp = p;  
17 s->nonzeros++;
```

C Implementation: Print Nonzeros

```
1 void printTerms(const Sparse *s)
2 {
3     printf("row col value\n");
4
5     for (int r = 0; r < ROWS; ++r) {
6         const Entry *p = s->rowHead[r];
7
8         while (p != NULL) {
9             printf("%d %d %d\n",
10                 p->row, p->col, p->value);
11             p = p->right;
12         }
13     }
14 }
```

The output follows row-major coordinate order.

C Implementation: Free the Matrix

```
1 void destroy(Sparse *s)
2 {
3     for (int r = 0; r < ROWS; ++r) {
4         Entry *p = s->rowHead[r];
5         while (p != NULL) {
6             Entry *next = p->right;
7             free(p);
8             p = next;
9         }
10        s->rowHead[r] = NULL;
11    }
12
13    for (int c = 0; c < COLS; ++c)
14        s->colHead[c] = NULL;
15
16    s->nonzeros = 0;
17 }
```

Free nodes through row lists only, then clear column heads

C Implementation: Example Driver

```
1 int main(void)
2 {
3     Sparse s = {0};
4     int terms[][3] = {
5         {0,2,7}, {1,0,5}, {1,4,9},
6         {2,3,-2}, {3,1,6}, {3,4,3}
7     };
8
9     for (int k = 0; k < 6; ++k) {
10         if (!setValue(&s, terms[k][0],
11                     terms[k][1], terms[k][2])) {
12             destroy(&s);
13             return EXIT_FAILURE;
14         }
15     }
16
17     printTerms(&s);
18     printf("A[1][4] = %d\n", getValue(&s, 1, 4));
```

C Program: Expected Output

```
row col value
0 2 7
1 0 5
1 4 9
2 3 -2
3 1 6
3 4 3
A[1][4] = 9
```

Combining the Program

Place the snippets in order and join the two setValue parts into one function.

Transpose of a Sparse Matrix

The transpose exchanges rows and columns:

$$A^T[j, i] = A[i, j].$$

For the running example:

$$A^T = \begin{bmatrix} 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ 0 & 9 & 0 & 3 \end{bmatrix}.$$

Dimensions change from 4×5 to 5×4 . The nonzero count remains 6.

Transpose in Triplet Form

Each entry changes as:

$$(i, j, v) \longrightarrow (j, i, v).$$

Row-major triplets of the transpose:

$$(0, 1, 5), \quad (1, 3, 6), \quad (2, 0, 7), \\ (3, 2, -2), \quad (4, 1, 9), \quad (4, 3, 3).$$

Ordering

Simply swapping row and column fields produces the correct coordinates, but not necessarily row-major order.

Simple Transpose Algorithm

For each original column:

- 1 Scan all triplets.
- 2 Select entries in that column.
- 3 Emit each as $(column, row, value)$.

For n columns and t nonzeros:

$$\text{Time} = O(nt + n).$$

Limitation

The same triplet array is scanned repeatedly. A counting-based transpose avoids those repeated scans.

Fast Transpose: Counting Entries

Count entries in each original column:

$$count = [1, 1, 1, 1, 2].$$

Find each transposed row's starting position:

$$start = [0, 1, 2, 3, 4].$$

- Original column 0 becomes transpose row 0.
- Original column 1 becomes transpose row 1.
- Each entry is placed directly in its output group.

$$\boxed{\text{Time} = O(n + t)}.$$

Fast Transpose Algorithm

```
1 FAST-TRANPOSE(entries, m, n, t)
2   count[0 .. n-1] = 0
3   for each (r, c, v) in entries
4       count[c] = count[c] + 1
5
6   position = 0
7   for c = 0 to n-1
8       next[c] = position
9       position = position + count[c]
10
11  allocate output array B of t entries
12  for each (r, c, v) in entries
13      k = next[c]
14      B[k] = (c, r, v)
15      next[c] = next[c] + 1
16
17  return B with dimensions n by m
```

Row-major input gives row-major output. Extra workspace is $O(n)$, excluding the output.

Transpose of an Orthogonal List

For a mutable orthogonal representation:

- ① For every node, exchange its row and column indices.
- ② Exchange its right and down links.
- ③ Exchange the row-head and column-head collections.
- ④ Exchange matrix dimensions.

Traversal Safety

Save the old next pointer before swapping links, or first collect the nodes to process.

Updating nodes takes $\Theta(t)$ time. Header handling depends on the chosen representation.

Adding Two Sparse Matrices

Addition requires equal dimensions:

$$C[i,j] = A[i,j] + B[i,j].$$

For sorted triplet arrays:

- Compare the current coordinates.
- Copy the entry with the smaller coordinate.
- At equal coordinates, add the values.
- Omit the result if the sum is zero.

$$\text{Time} = O(t_A + t_B)$$

Assumes output space is available and input triplets are canonical.

Sparse Addition: Worked Example

$$A = \begin{bmatrix} 0 & 4 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 5 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & -4 & 7 \\ 0 & 0 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

$$A + B = \begin{bmatrix} 0 & 0 & 7 \\ 2 & 0 & 0 \\ 0 & 0 & 8 \end{bmatrix}.$$

Result triplets:

$$(0, 2, 7), \quad (1, 0, 2), \quad (2, 2, 8).$$

Compressed Sparse Row: CSR

CSR uses three arrays:

- ① values: nonzero values in row order.
- ② colIndex: column of each stored value.
- ③ rowPtr: starting offsets of rows.

For row i , its entries occupy:

$$\boxed{\text{rowPtr}[i] \leq k < \text{rowPtr}[i + 1]}.$$

Array Lengths

values and colIndex have length t .

rowPtr has length $m + 1$, with $\text{rowPtr}[m] = t$.

CSR for the Running Example

$$values = [7, 5, 9, -2, 6, 3]$$

$$colIndex = [2, 0, 4, 3, 1, 4]$$

$$rowPtr = [0, 1, 3, 4, 6]$$

For row 1:

$$rowPtr[1] = 1, \quad rowPtr[2] = 3.$$

Therefore its entries use offsets 1 and 2:

$$A[1, 0] = 5, \quad A[1, 4] = 9.$$

Compressed Sparse Column: CSC

CSC stores entries grouped by column:

- `values`.
- `rowIndex`.
- `colPtr`.

For the running example:

$$values = [5, 6, 7, -2, 9, 3]$$
$$rowIndex = [1, 3, 0, 2, 1, 3]$$
$$colPtr = [0, 1, 2, 3, 4, 6].$$

Column j occupies offsets $colPtr[j]$ through $colPtr[j + 1] - 1$.

Sparse Matrix-Vector Multiplication

For:

$$y = Ax,$$

compute:

$$y_i = \sum_{j=0}^{n-1} A[i,j]x_j.$$

Only nonzero entries contribute.

Row-Based Algorithm

Initialize $y_i = 0$. For each stored (i, j, v) in row i , add vx_j to y_i .

Using row lists or CSR:

$$\text{Time} = O(m + t).$$

CSR Matrix-Vector Multiplication

```
1 SPARSE-MATVEC(values, colIndex, rowPtr, x, m)
2   for i = 0 to m-1
3     y[i] = 0
4
5     for k = rowPtr[i] to rowPtr[i+1]-1
6       j = colIndex[k]
7       y[i] = y[i] + values[k] * x[j]
8
9   return y
```

Empty Rows

An empty row has:

$$rowPtr[i] = rowPtr[i + 1].$$

Its inner loop performs no iterations.

Representation Comparison

Format	Space	Strength	Limitation
Dense	$\Theta(mn)$	Direct access	Stores all zeros
Triplet	$\Theta(t)$	Simple entry storage	Packed-array updates may shift entries
Orthogonal	$\Theta(t + m + n)$	Row and column traversal	Pointer overhead
CSR	$\Theta(t + m)$	Compact row access	Column access and structural updates
CSC	$\Theta(t + n)$	Compact column access	Row access and structural updates

Worked Problem: Storage Count

A 1000×1000 matrix has 5000 nonzero entries.

Dense storage requires:

$$1000 \times 1000 = 1,000,000$$

value slots.

Triplet storage requires:

$$3(5000) = 15,000$$

scalar fields, excluding metadata.

CSR requires:

$$5000 + 5000 + 1001 = \boxed{11,001}$$

value/index fields.

These are field counts. Byte counts depend on the sizes of values and indices.

Worked Problem: Matrix-Vector Product

For the running matrix, let:

$$x = [1 \quad 2 \quad 3 \quad 4 \quad 5]^T.$$

Then:

$$y_0 = 7(3) = 21,$$

$$y_1 = 5(1) + 9(5) = 50,$$

$$y_2 = -2(4) = -8,$$

$$y_3 = 6(2) + 3(5) = 27.$$

$$y = [21, 50, -8, 27]^T.$$

Applications

- Graph adjacency matrices.
- Scientific and engineering computations.
- Finite-element and network models.
- Document-term matrices.
- User-item interaction matrices.
- Student-course registration systems.

Connection

Multilists represent multiple relationships. Orthogonal sparse matrices use this idea for row and column membership.

Practice Matrix

Consider:

$$B = \begin{bmatrix} 0 & 8 & 0 & 0 \\ 0 & 0 & 0 & 5 \\ 2 & 0 & 0 & 0 \end{bmatrix}.$$

- 1 Find its density.
- 2 Write its triplet representation.
- 3 Draw its row and column lists.
- 4 Write its CSR arrays.
- 5 Find its transpose.

Practice Answers: Triplets and Density

There are:

$$m = 3, \quad n = 4, \quad t = 3.$$

Density:

$$\frac{3}{12} = 25\%.$$

Triplets:

$$(0, 1, 8), \quad (1, 3, 5), \quad (2, 0, 2).$$

Metadata:

$$(rows, columns, nonzeros) = (3, 4, 3).$$

Practice Answers: CSR and Transpose

CSR:

$$\begin{aligned} \text{values} &= [8, 5, 2], \\ \text{colIndex} &= [1, 3, 0], \\ \text{rowPtr} &= [0, 1, 2, 3]. \end{aligned}$$

Transpose:

$$B^T = \begin{bmatrix} 0 & 0 & 2 \\ 8 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 5 & 0 \end{bmatrix}.$$

Practice Answers: Orthogonal Lists

Row lists:

$$R_0 \longrightarrow (0, 1, 8) \longrightarrow \text{NULL},$$

$$R_1 \longrightarrow (1, 3, 5) \longrightarrow \text{NULL},$$

$$R_2 \longrightarrow (2, 0, 2) \longrightarrow \text{NULL}.$$

Column lists:

$$C_0 \longrightarrow (2, 0, 2) \longrightarrow \text{NULL},$$

$$C_1 \longrightarrow (0, 1, 8) \longrightarrow \text{NULL},$$

$$C_2 \longrightarrow \text{NULL},$$

$$C_3 \longrightarrow (1, 3, 5) \longrightarrow \text{NULL}.$$

Quick Revision

Concept	Key Point
Multilist	Shared records participate in several lists
Sparse matrix	Relatively few nonzero entries
Triplet	Row, column and value
Orthogonal list	One entry in a row list and a column list
Right link	Next nonzero in the same row
Down link	Next nonzero in the same column
CSR	Values, column indices and row offsets
CSC	Values, row indices and column offsets
Transpose	Exchange row and column roles
Deletion	Unlink from both lists before freeing

References

- Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed. *Fundamentals of Data Structures in C*. Topics: sparse matrices and linked representations.
- Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman. *Data Structures and Algorithms*. Topics: linked structures and multilists.
- OpenDSA. *The Sparse Matrix*. Orthogonal linked-list representation.

Implementation Convention

These slides use separate row and column header arrays with NULL-terminated lists.

Thank You